Chapter 3. API Contents

Develop code that uses the primitive wrapper classes (such as **Boolean**, **Character**, **Double**, **Integer**, etc.), and/or autoboxing & unboxing. Discuss the differences between the **String**, **StringBuilder**, and **StringBuffer** classes.

Autoboxing/unboxing of primitive types

Manual conversion between primitive types (such as an `int`) and wrapper classes (such as `Integer`) is necessary when adding a primitive data type to a collection. As an example, consider an `int` being stored and then retrieved from an `ArrayList`:

```
list.add(0, new Integer(59));
int n = ((Integer)(list.get(0))).intValue();
```

The new autoboxing/unboxing feature eliminates this manual conversion. The above segment of code can be written as:

```
list.add(0, 59);
int total = list.get(0);
```

However, note that the wrapper class, `Integer` for example, must be used as a generic type:

```
List<Integer> list = new ArrayList<Integer>();
```

The autoboxing and auto-unboxing of Java primitives produces code that is more concise and easier to follow.

```
int i = 10;

Integer iRef = new Integer(i);  // Explicit Boxing
int j = iRef.intValue();        // Explicit Unboxing

iRef = i;       // Automatic Boxing
j = iRef;       // Automatic Unboxing
```

Automatic boxing and unboxing conversions alleviate the drudgery in converting values of primitive types to objects of the corresponding wrapper classes and vice versa.

Boxing conversion converts primitive values to objects of corresponding wrapper types: if `p` is a value of a `primtiveType`, boxing conversion converts `p` into a reference `ref` of corresponding `WrapperType`, such that `ref.primitiveTypeValue() == p`.

Unboxing conversion converts objects of wrapper types to values of corresponding primitive types: if `ref` is a reference of a `WrapperType`, unboxing conversion converts the reference `ref` into `ref.primitiveTypeValue()`, where `primitiveType` is the primitive type corresponding to the `WrapperType`.

Assignment conversions on `boolean` and numeric types:

```
boolean boolVal = true;
byte b = 2;
short s = 2;
char c ='2';
int i = 2;

// Boxing
Boolean boolRef = boolVal;
Byte bRef = 88;
Short sRef = 2;
Character cRef = '2';
Integer iRef = 2;
// Integer iRef1 = s; // WRONG ! short not assignable to Integer

// Unboxing
boolean boolVal1 = boolRef;
byte b1 = bRef;
short s1 = sRef;
char c1 = cRef;
int i1 = iRef;
```

Method invocation conversions on actual parameters:

```
...
flipFlop("(String, Integer, int)", new Integer(4), 2004);
...
private static void flipFlop(String str, int i, Integer iRef) {
        out.println(str + " ==> (String, int, Integer)");
}
```

The output:

```
(String, Integer, int) ==> (String, int, Integer)
```

Casting conversions:

```
Integer iRef = (Integer) 2;     // Boxing followed by identity cast
int i = (int) iRef;             // Unboxing followed by identity cast
// Long lRef = 2;               // WRONG ! Type mismatch: cannot convert from int to Long
// Long lRef = (Long) 2;        // WRONG ! int not convertible to Long
Long lRef_1 = (long) 2;         // OK ! explicit cast to long
Long lRef_2 = 2L;               // OK ! long literal
```

Numeric promotion: unary and binary:

```
Integer iRef = 2;
long l1 = 2000L + iRef;     // binary numeric promotion
int i = -iRef;              // unary numeric promotion
```

In the `if` statement, condition can be `Boolean`:

```

Boolean expr = true;
```

```
if (expr) {
        out.println(expr);
} else {
        out.println(!expr); // Logical complement operator
}
```

In the `switch` statement, the switch expression can be `Character`, `Byte`, `Short` or `Integer`:

```
// Constants
final short ONE = 1;
final short ZERO = 0;
final short NEG_ONE = -1;

// int expr = 1; // (1) short is assignable to int. switch works.
// Integer expr = 1; // (2) short is not assignable to Integer. switch compile error.
Short expr = 1; // (3) OK. Cast is not required.
switch (expr) { // (4) expr unboxed before case comparison.
        case ONE:
                out.println("ONE"); break;
        case ZERO:
                out.println("ZERO"); break;
        case NEG_ONE:
                out.println("NEG_ONE"); break;
        default:
                assert false;
}
```

The output:

```
ONE
```

In the `while`, `do-while` and `for` statements, the condition can be `Boolean`:

```
Boolean expr = true;
while (expr) {
        expr = !expr;
}
```

```
Character[] version = { '5', '.', '0' };    // Assignment: boxing
for (Integer iRef = 0;                      // Assignment: boxing
        iRef < version.length;           // Comparison: unboxing
        ++iRef) {                           // ++: unboxing and boxing
    out.println(iRef + ": " + version[iRef]); // Array index: unboxing
}
```

Output:

```
0: 5
1: .
2: 0
```

Boxing and unboxing in collections/maps:

```
String[] words = new String[] {"aaa", "bbb", "ccc", "aaa"};
Map<String, Integer> m = new TreeMap<String, Integer>();
for (String word : words) {
        Integer freq = m.get(word);
        m.put(word, freq == null ? 1 : freq + 1);
}
out.println(m);
```

The output:

```
{aaa=2, bbb=1, ccc=1}
```

In the next example an `int` is being stored and then retrieved from an `ArrayList`. The J2SE 5.0 leaves the conversion required to transition to an `Integer` and back to the compiler.

Before:

```
ArrayList<Integer> list = new ArrayList<Integer>();
list.add(0, new Integer(42));
int total = (list.get(0)).intValue();
```

After:

```
ArrayList<Integer> list = new ArrayList<Integer>();
list.add(0, 42);
int total = list.get(0);
```

An `Integer` expression can have a `null` value. If your program tries to autounbox `null`, it will throw a `NullPointerException`. The `==` operator performs reference identity comparisons on `Integer` expressions and value equality comparisons on `int` expressions. Finally, there are performance costs associated with boxing and unboxing, even if it is done automatically.

Here is another sample program featuring autoboxing and unboxing. It is a static factory that takes an `int` array and returns a `List` of `Integer` backed by the array. This method provides the full richness of the `List` interface atop an `int` array. All changes to the list write through to the array and vice-versa:

```
// List adapter for primitive int array
public static List<Integer> asList(final int[] a) {
        return new AbstractList<Integer>() {

                public Integer get(int i) { return a[i]; }

                // Throws NullPointerException if val == null
                public Integer set(int i, Integer val) {
                        Integer oldVal = a[i];
                        a[i] = val;
                        return oldVal;
                }

                public int size() { return a.length; }
```

```
        };
}
```

Wrappers and primitives comparison:

```
public class WrappersTest {
        public static void main(String[] s) {
                Integer i1 = new Integer(2);
                Integer i2 = new Integer(2);
                System.out.println(i1 == i2); // FALSE

                Integer j1 = 2;
                Integer j2 = 2;
                System.out.println(j1 == j2); // TRUE

                Integer k1 = 150;
                Integer k2 = 150;
                System.out.println(k1 == k2); // FALSE

                Integer jj1 = 127;
                Integer jj2 = 127;
                System.out.println(jj1 == jj2); // TRUE

                int jjj1 = 127;
                Integer jjj2 = 127;
                System.out.println(jjj1 == jjj2); // TRUE

                Integer kk1 = 128;
                Integer kk2 = 128;
                System.out.println(kk1 == kk2); // FALSE

                Integer kkk1 = 128;
                int kkk2 = 128;
                System.out.println(kkk1 == kkk2); // TRUE

                Integer w1 = -128;
                Integer w2 = -128;
                System.out.println(w1 == w2); // TRUE

                Integer m1 = -129;
                Integer m2 = -129;
                System.out.println(m1 == m2); // FALSE

                int mm1 = -129;
                Integer mm2 = -129;
                System.out.println(mm1 == mm2); // TRUE
        }
}
```

NOTE, certain primitives are always to be boxed into the same immutable wrapper objects. These objects are then CACHED and REUSED, with the expectation that these are commonly used objects. These special values are:

- The `boolean` values `true` and `false`.
- The `byte` values.
- The `short` and `int` values between `-128` and `127`.
- The `char` values in the range '`\u0000`' to '`\u007F`'.

```
Character c1 = '\u0000';
Character c2 = '\u0000';
System.out.println("c1 == c2 : " + (c1 == c2)); // TRUE !!!

Character c11 = '\u00FF';
Character c12 = '\u00FF';
System.out.println("c11 == c12 : " + (c11 == c12)); // FALSE
```

```
c1 == c2 : true
c11 == c12 : false
```

The following example gives `NullPointerException`:

```
Integer i = null;
int j = i; // java.lang.NullPointerException !!!
```

This example demonstrates methods resolution when selecting overloaded method:

```
public static void main(String[] args) {
        doSomething(1);
        doSomething(2.0);
}

public static void doSomething(double num) {
        System.out.println("double : " + num);
}

public static void  doSomething(Integer num) {
        System.out.println("Integer : " + num);
}
```

The output is:

```
double : 1.0
double : 2.0
```

Java 5.0 will always select the same method that would have been selected in Java 1.4.

The method resolution inludes three passes:

1. Attempt to locate the correct method WITHOUT any boxing, unboxing, or vararg invocations.
2. Attempt to locate the correct method WITH boxing, unboxing, and WITHOUT any vararg invocations.
3. Attempt to locate the correct method WITH boxing, unboxing, or vararg invocations.

`String`

The `String` class represents character strings. All string literals in Java programs, such as "abc", are implemented as instances of this class.

All implemented interfaces: `Serializable`, `CharSequence`, `Comparable<String>`.

This method returns the number of characters in the string as in:

```
int length ()
```

This example results in variable x holding the value 8:

```
String str = "A string";
int x = str.length ();
```

Removes whitespace from the leading and trailing edges of the string:

```
String trim ()
```

This results in the variable str referencing the string "14 units":

```
String string = "  14 units  ";
String str = string.trim ();
```

The following methods return the index, starting from 0, for the location of the given character in the string. (The char value will be widened to int):

```
int indexOf (int ch)
int lastIndexOf (int ch)
```

For example:

```
String string = "One fine day";
int x = string.indexOf ('f');
```

This results in a value of 4 in the variable x. If the string holds NO such character, the method returns -1.

To continue searching for more instances of the character, you can use the method:

```
indexOf(int ch, int fromIndex)
```

This will start the search at the fromIndex location in the string and search to the end of the string.

The methods:

```
indexOf (String str)
indexOf (String str, int fromIndex)
```

provide similar functions but search for a sub-string rather than just for a single character.

Similarly, the methods:

```
lastIndexOf (int ch)
lastIndexOf (int ch, int fromIndex)

lastIndexOf (String str)
lastIndexOf (String str, int fromIndex)
```

search backwards for characters and strings starting from the right side and moving from right to left. (The `fromIndex` second parameter still counts from the left, with the search continuing from that index position toward the beginning of the string).

These two methods test whether a string begins or ends with a particular substring:

```
boolean startsWith (String prefix)
boolean endsWith (String str)
```

For example:

```
String [] str = {"Abe", "Arthur", "Bob"};
for (int i=0; i < str.length (); i++) {
        if (str1.startsWith ("Ar")) doSomething ();
}
```

The first method return a new string with all the characters set to lower case while the second returns the characters set to upper case:

```
String toLowerCase ()
String toUpperCase ()
```

Example:

```
String [] str = {"Abe", "Arthur", "Bob"};
for (int i=0; i < str.length(); i++){
    if (str1.toLowerCase ().startsWith ("ar")) doSomething ();
}
```

StringBuilder

J2SE5.0 added the `StringBuilder` class, which is a drop-in replacement for `StringBuffer` in cases where thread safety is not an issue. Because `StringBuilder` is NOT synchronized, it offers FASTER performance than `StringBuffer`.

In general, you should use `StringBuilder` in preference over `StringBuffer`. In fact, the J2SE 5.0 `javac` compiler normally uses `StringBuilder` instead of `StringBuffer` whenever you perform string concatenation as in:

```
System.out.println ("The result is " + result);
```

All the methods available on `StringBuffer` are also available on `StringBuilder`, so it really is a drop-in replacement.

Instances of `StringBuilder` are not safe for use by multiple threads. If such synchronization is required then it is recommended that `StringBuffer` be used.

StringBuffer

String objects are immutable, meaning that once created they cannot be altered. Concatenating two strings does not modify either string but instead creates a new string object:

```
String str = "This is";
str = str + " a new string object";
```

Here `str` variable now references a completely new object that holds the "`This is a new string object`" string.

This is not very efficient if you are doing extensive string manipulation with lots of new strings created through this sort of append operations. The `String` class maintains a pool of strings in memory. String literals are saved there and new strings are added as they are created. Extensive string manipulation with lots of new strings created with the `String` append operations can therefore result in lots of memory taken up by unneeded strings. Note however, that if two string literals are the same, the second string reference will point to the string already in the pool rather than create a duplicate.

The class `java.util.StringBuffer` offers more efficient string creation. For example:

```
StringBuffer strb = new StringBuffer ("This is");
strb.append(" a new string object");
System.out.println (strb.toString());
```

The `StringBuffer` uses an internal `char` array for the intermediate steps so that new strings objects are not created. If it becomes full, the array is copied into a new larger array with the additional space available for more append operations.

The `StringBuffer` is a thread-safe, mutable sequence of characters. A string buffer is like a `String`, but can be modified. At any point in time it contains some particular sequence of characters, but the length and content of the sequence can be changed through certain method calls.

String buffers are safe for use by multiple threads. The methods are synchronized where necessary so that all the operations on any particular instance behave as if they occur in some serial order that is consistent with the order of the method calls made by each of the individual threads involved.

The principal operations on a `StringBuffer` are the `append` and `insert` methods, which are overloaded so as to accept data of any type. Each effectively converts a given datum to a string and then appends or inserts the characters of that string to the string buffer. The `append` method always adds these characters at the end of the buffer; the `insert` method adds the characters at a specified point.

For example, if `z` refers to a string buffer object whose current contents are "`start`", then the method call `z.append("le")` would cause the string buffer to contain "`startle`", whereas `z.insert(4, "le")` would alter the string buffer to contain "`starlet`".

In general, if `sb` refers to an instance of a `StringBuffer`, then `sb.append(x)` has the same effect as `sb.insert(sb.length(), x)`.

Whenever an operation occurs involving a source sequence (such as appending or inserting from a source sequence) this class synchronizes only on the string buffer performing the operation, not on the source.

Every string buffer has a capacity. As long as the length of the character sequence contained in the string buffer does not exceed the capacity, it is not necessary to allocate a new internal buffer array. If the internal buffer overflows, it is automatically made larger. As of release JDK 5, this class has been supplemented with an equivalent class designed for use by a single thread, `StringBuilder`. The `StringBuilder` class should generally be used in preference to this one, as it supports all of the same operations but it is faster, as it performs no synchronization.

`StringBuffer` class DOES NOT override the `equals()` method. Therefore, it uses `Object` class' `equals()`, which only checks for equality of the object references. `StringBuffer.equals()` does not return `true` even if the two `StringBuffer` objects have the same contents:

```
StringBuffer sb = new StringBuffer("ssss");
StringBuffer sb_2 = new StringBuffer("ssss");
out.println("sb equals sb_2 : " + sb.equals(sb_2));
```

The output:

```
sb equals sb_2 : false
```

NOTE, `String`'s `equals()` method checks if the argument if of type string, if not it returns `false`:

```
StringBuffer sb = new StringBuffer("ssss");
String st = new String("ssss");
out.println("sb equals st : " + sb.equals(st)); // Always 'false'
out.println("st equals sb : " + st.equals(sb)); // Always 'false'
```

The output:

```
sb equals st : false
st equals sb : false
```

Prev

Recognize situations that will result in any of the following being thrown: ArrayIndexOutOfBoundsException, ClassCastException, IllegalArgumentException, IllegalStateException, NullPointerException, NumberFormatException, AssertionError, ExceptionInInitializerError, StackOverflowError or NoClassDefFoundError. Understand which of these are thrown by the virtual machine and recognize situations in which others should be thrown programatically.

Up

Home

Next

Given a scenario involving navigating file systems, reading from files, or writing to files, develop the correct solution using the following classes (sometimes in combination), from java.io: BufferedReader, BufferedWriter, File, FileReader, FileWriter and PrintWriter.